Architecture 1001: x86-64 Assembly ArrayLocalVariable2.c

Xeno Kovah – 2021

xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

ArrayLocalVariable2.c

Zero-initializing the array

```c
short main(){
    int a;
    short b[64] = {0};
    a = 0x5eaf00d;
    b[1] = (short)a;
    return b[1];
}
```

```
main:
0000000140001000  push        rdi
0000000140001002  sub         rsp,90h
0000000140001009  lea         rax,[rsp+10h]
000000014000100E  mov         rdi,rax
0000000140001011  xor         eax,eax
0000000140001013  mov         ecx,80h
0000000140001018  rep stos    byte ptr [rdi]
000000014000101A  mov         dword ptr [rsp],5EAF00Dh
0000000140001021  mov         eax,2
0000000140001026  imul        rax,rax,1
000000014000102A  movzx       ecx,word ptr [rsp]
000000014000102E  mov         word ptr [rsp+rax+10h],cx
0000000140001033  mov         eax,2
0000000140001038  imul        rax,rax,1
000000014000103C  movzx       eax,word ptr [rsp+rax+10h]
0000000140001041  add         rsp,90h
0000000140001048  pop         rdi
0000000140001049  ret
```

ArrayLocalVariable2.c

Zero-initializing the array

```
short main(){
    int a;
    short b[64] = {0};
    a = 0x5eaf00d;
    b[1] = (short)a;
    return b[1];
}
```



```
main:
0000000140001000  push        rdi
0000000140001002  sub         rsp,90h
0000000140001009  lea         rax,[rsp+10h]
000000014000100E  mov         rdi,rax
0000000140001011  xor         eax,eax
0000000140001013  mov         ecx,80h
0000000140001018  rep stos    byte ptr [rdi]
000000014000101A  mov         dword ptr [rsp],5EAF00Dh
0000000140001021  mov         eax,2
0000000140001026  imul        rax,rax,1
000000014000102A  movzx       ecx,word ptr [rsp]
000000014000102E  mov         word ptr [rsp+rax+10h],cx
0000000140001033  mov         eax,2
0000000140001038  imul        rax,rax,1
000000014000103C  movzx       eax,word ptr [rsp+rax+10h]
0000000140001041  add         rsp,90h
0000000140001048  pop         rdi
0000000140001049  ret
```

REP STOS
Repeat **Sto**re **S**tring

- STOS is one of number of instructions that can have the “rep” prefix added to it, which repeat a single instruction multiple times.
- All rep operations use *cx register as a “counter” to determine how many times to loop through the instruction. Each time it executes, it decrements *cx by 1. Once *cx == 0, it continues to the next instruction.
- Either stores 1, 2, 4, or 8 bytes at a time
- Either fill 1 byte at [di] with al or fill 1/2/4/8 bytes at [*di] with al/*ax

REP STOS
Repeat **Sto**re **S**tring

- Increments the *di register by 1/2/4/8 bytes at a time, so that the repeated store operation is storing into consecutive locations.
- So there are 3 pieces which must happen before the actual rep stos occurs:
  - Set *di to the start destination,
  - Set *ax/al to the value to store,
  - Set *cx to the number of times to store

REP STOS
Repeat **Sto**re **S**tring

- STOS is one of number of instructions that can have the “rep” prefix added to it, which repeat a single instruction multiple times.
- 64 bit rep operations use rcx register as a “counter” to determine how many times to loop through the instruction. Each time it executes, it decrements rcx by 1. Once rcx == 0, it continues to the next instruction.
- Either stores 1, 2, 4, or 8 bytes at a time
- Fill 1/2/4/8 bytes at [rdi] with al/ax/eax/rax
- Increment the rdi register by 1/2/4/8 bytes at a time, so that the repeated store operation is storing into consecutive locations.
- So there are 3 pieces which must happen before the actual rep stos occurs:
  - Set rdi to the start destination
  - Set rax/al to the value to store
  - Set rcx to the number of times to store

Book page 101-102

VS vs. GCC et al.
Caller vs. Callee-save Regs

VS Caller-saved	RAX		RCX	RDX				R8-11	
VS Callee-saved		RBX			RSI	RDI	RBP		R12-15
System V (GCC et al.) Caller-saved	RAX		RCX	RDX	RSI	RDI		R8-11	
System V (GCC et al.) Callee-saved		RBX					RBP		R12-15



STOP

&&

STEP THROUGH

THE ASSEMBLY

&&

CHECK YOUR UNDERSTANDING



STOP

&&

STEP THROUGH

THE ASSEMBLY

&&

CHECK YOUR UNDERSTANDING

ArrayLocalVariable2.c takeaways

- If you're manually coding asm, REP STOS is functionally a memset()
- Sometimes when you use memset() from C, the compiler may turn it into a REP STOS

```
short main(){
    int a;
    short b[64] = {0};
    a = 0x5eaf00d;
    b[1] = (short)a;
    return b[1];
}
```

```
main:
0000000140001000  push        rdi
0000000140001002  sub         rsp,90h
0000000140001009  lea         rax,[rsp+10h]
000000014000100E  mov         rdi,rax
0000000140001011  xor         eax,eax
0000000140001013  mov         ecx,80h
0000000140001018  rep stos    byte ptr [rdi]
000000014000101A  mov         dword ptr [rsp],5EAF00Dh
0000000140001021  mov         eax,2
0000000140001026  imul        rax,rax,1
000000014000102A  movzx       ecx,word ptr [rsp]
000000014000102E  mov         word ptr [rsp+rax+10h],cx
0000000140001033  mov         eax,2
0000000140001038  imul        rax,rax,1
000000014000103C  movzx       eax,word ptr [rsp+rax+10h]
0000000140001041  add         rsp,90h
0000000140001048  pop         rdi
0000000140001049  ret
```

Instructions we now know (29)

- NOP
- PUSH/POP
- CALL/RET
- MOV
- ADD/SUB
- IMUL
- MOVZX/MOVSX
- LEA
- JMP/Jcc (family)
- CMP/TEST
- AND/OR/XOR/NOT
- INC/DEC
- SHR/SHL/SAR/SAL
- DIV/IDIV
- **REP STOS**

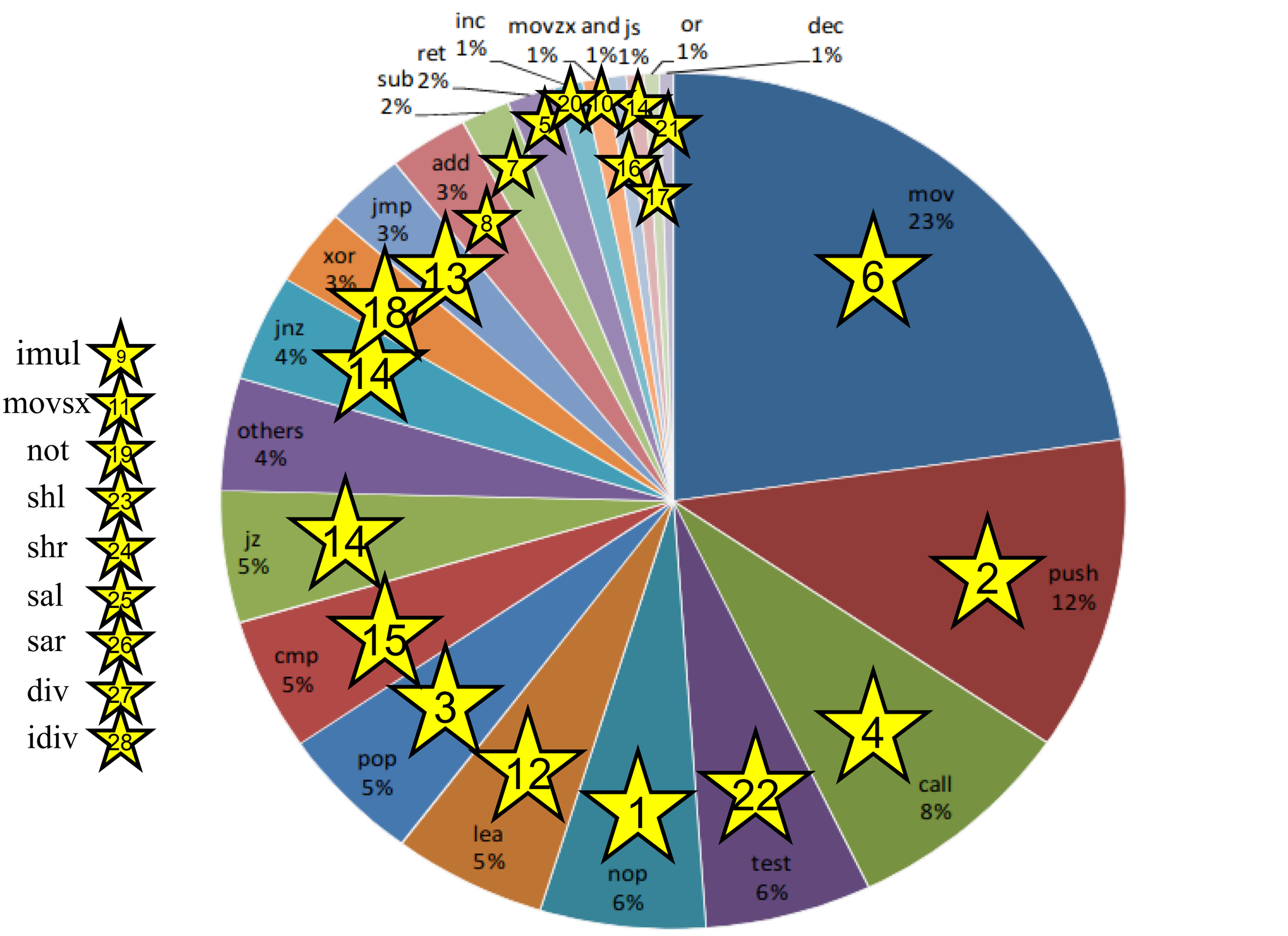


Fig. 2 Web browsers' instruction frequencies

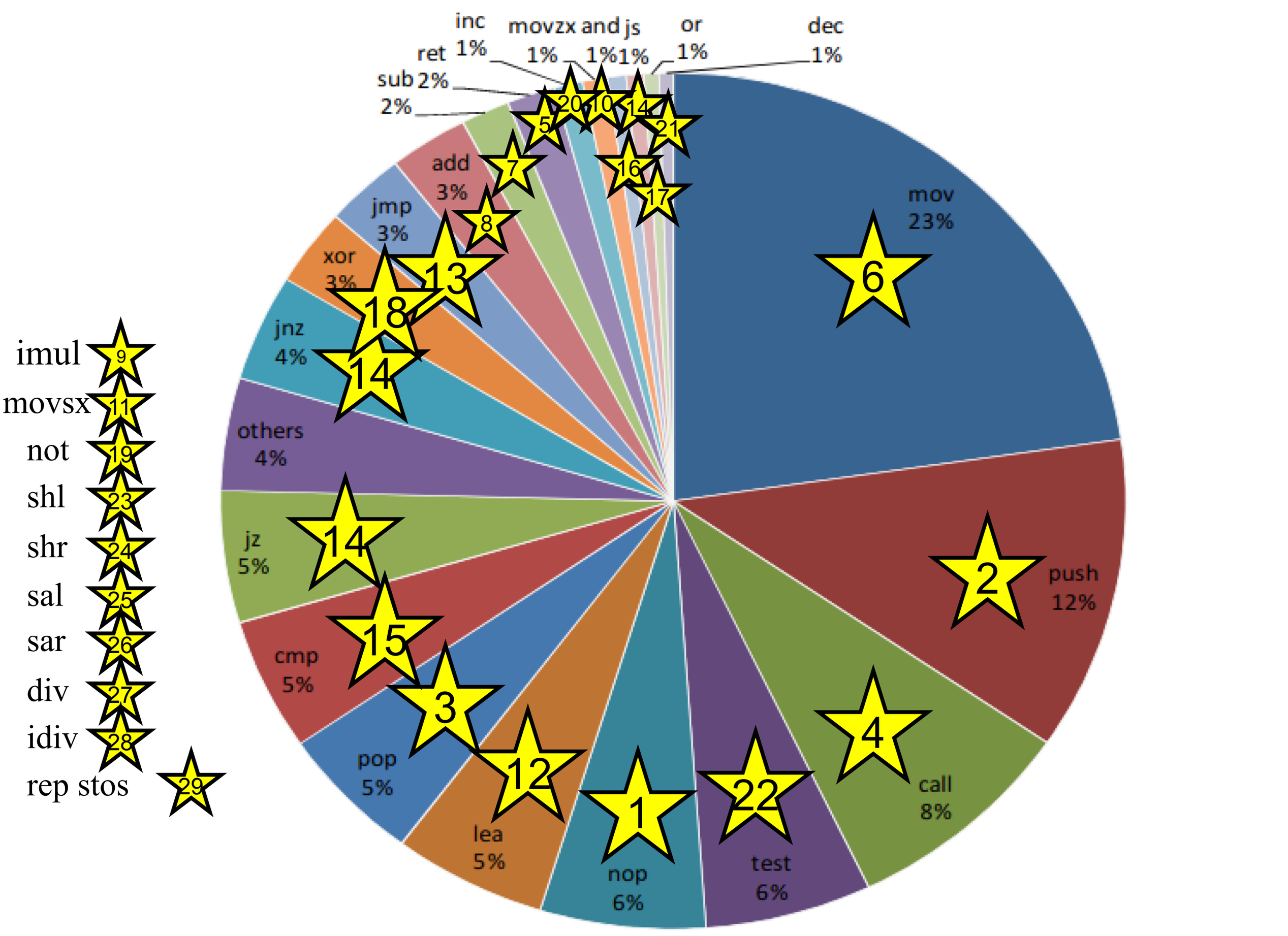


Fig. 2 Web browsers' instruction frequencies